

Real-Time Song Generation: A Csound API Case Study

Garrett Eckl¹, Steven Yi, Edward Costello and Lex Van Der Ploeg²

¹ McNeese State University

² VDP II LLC

Abstract. This case study presents RIFFIT, a commercial text-to-song platform that utilizes the Csound audio programming language. This platform enables users to turn any text into a fully-produced song in near real-time, with a variety of genres and voice models to select from. Csound serves as the platform’s primary synthesis engine, performing a custom CSD file that is dynamically generated based on both the inputted text and selected musical preferences.

This paper begins with a brief introduction to the field of automatic music generation, which is followed by an overview of the RIFFIT platform and key developments that have occurred to date. The software’s architecture is then outlined, with topics including the dynamic generation of Csound code, the manual performance of Csound files via the Csound API, and strategies for the web deployment of Csound-enabled software. Lastly, an analysis of Csound’s suitability for such commercial applications is provided.

Keywords: Csound API, Music Generators, Node.js, Algorithmic Composition

1 Introduction

The field of automatic music generation has seen significantly renewed interest in recent years, primarily through the development of AI-generated music systems. However, questions regarding the legal and ethical implications of these systems have raised concerns about their artistic value and marketability [1]. These concerns support the need for generative music tools that do not rely on deep learning techniques and unverified training data.

RIFFIT is a commercial text-to-song platform consisting of a backend system for real-time song generation and a variety of frontends, including web applications and an application programming interface. Originally developed as a learning tool to help people with neurological differences improve reading retention and comprehension, additional applications for entertainment and communication were soon introduced, with over one million custom songs generated in the following year [2]. Users enter lyrics into the system and select from a range of musical preferences, including genre and voice.

Unlike many AI-generated music systems that create music by converting a textual input directly into a fully-produced audio output with machine learning, RIFFIT implements these methods with the Csound audio programming language and singing voice synthesizers developed by Dreamtonics [3], using patent-protected techniques to intelligently generate musical scores based on the user’s input. The following section of this paper will explore the architecture of this software as it relates to Csound. This includes designs for dynamically compiling Csound orchestras and scores, methods for performing Csound documents with external audio input, and strategies for deploying such platforms as web applications.

2 Software Architecture

RIFFIT’s text-to-song backend service generates a song from an inputted text and various musical parameters, requiring at minimum the following:

- 1) *Text* (the song’s lyrics)
- 2) *Genre*
- 3) *Voice* (selected singing voice synthesizer model)
- 4) *Tempo*

The backend comprises three main components. First, the Song Composer dynamically generates both a vocal melody and accompanying Csound-based instrumentation. Second, the Song Engine

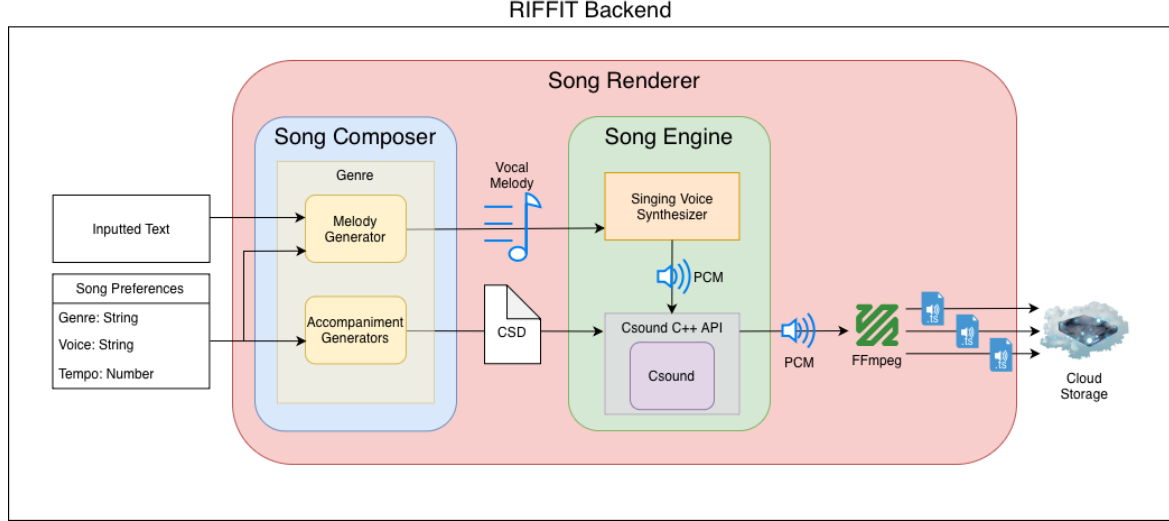


Fig. 1. RIFFIT Backend Architecture

renders both the Csound accompaniment and the singing synthesizer vocal melody. Third, the Song Renderer segments the raw audio and uploads the chunks to a storage platform for web streaming. See Figure 1 for an outline of this architecture.

2.1 Song Composer

The Song Composer serves as an automatic composition system. Given the user’s selected musical preferences and inputted text, this module produces both a vocal melody and an accompanying instrumental score in the form of a unified Csound data file. Given the complex nature of symbolic music representation, the Song Composer is written in TypeScript for its static typing and general ease of use. TypeScript classes are created to represent abstractions of musical elements, each of which possess a method for dynamically generating a corresponding snippet of Csound code. At the top level, a *Genre* class provides both a predesigned Csound orchestra template and a selection of accompanying Csound score generators.

The orchestra template constructs the “sound” of the genre by defining a unique configuration of instrument, effect, and audio bus components. A typical orchestra includes anywhere from one to a dozen instruments, in addition to a voice instrument that receives input signal from the singing voice synthesizer (with audio written to the audio input working buffer and read using the *inch* opcode [4]). Audio buses are used to route instruments together through effects units, all of which are then routed to the master bus. Control channels are created to set genre-specific default values for volume, panning, and other relevant component parameters. See Figure 2 for an example orchestra configuration.

Each genre also possesses a selection of score generators, which provide instructions for intelligently performing the orchestra’s instruments given the stylistic considerations of the genre and a unifying song structure. These accompaniment generators are sorted into one or more categories (bass, percussion, rhythm, lead, etc.) and a unique combination is selected from each. Every accompaniment generator returns an array of Csound i-statements, taking into account things such as the current section (verse, chorus, bridge, etc.), the current beat, and the scale degrees of the current chord.

Finally, a vocal melody is created for the user-inputted text. This melody is informed by stylistic considerations of the selected genre, the unifying song structure, and the natural prosody of the text. The user’s selected voice also influences the melody, restricting the pitch range and determining vocal inflections.

2.2 Song Engine

The Song Engine serves as a processor. It is responsible for compiling and performing the Csound code, as well as rendering the singing voice synthesizer. Given the requirement of near real-time song

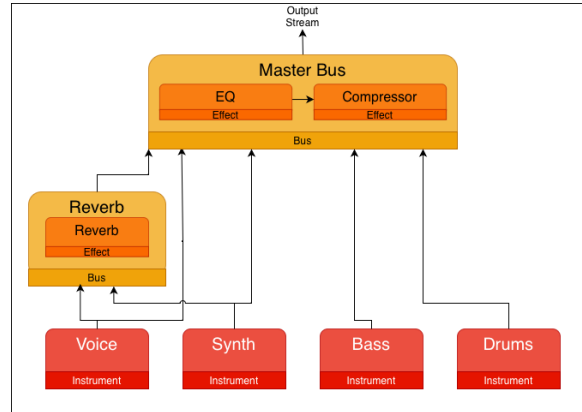


Fig. 2. Example Genre Signal Flow

generation, this module is written in C++ and utilizes the Csound C++ API. To process a full song, the engine executes the following steps.

First, the singing voice synthesizer models are loaded and Csound is initialized with its default signal handling and exit handler disabled. A Csound instance is created, and the default I/O and real time audio modules are disabled. These customizations allow the Song Engine to fully handle the execution of the Csound document created by the Song Composer. This document is then compiled, and the genre's default control channel values are used to set the control channels in Csound. Some of these default values may be overridden by user customizations to tempo, voice volume, etc.

Next, the vocal melody data is passed into the singing voice synthesizer, which begins rendering audio asynchronously. A single-producer single-consumer (SPSC) queue is used to pass generated samples from the singing voice synthesizer to Csound [5]. This architecture allows for each part to run in its own thread and optimizes overall processing times. When a ksmps number of audio samples have been added to the queue, they are dequeued into the Csound audio input working buffer (SPIN) and a ksmps worth of audio samples are performed. A callback function then returns these samples to the parent process and SPIN is cleared. This process is repeated until the entire score has been rendered.

Piping audio from singing voice synthesizer into Csound (C++ code)

```
while (csoundPerformKsmps(csound) == 0) {
    // copy Csound spout and return frame to callback
    for (int i = 0; i < ksmps * outputChannels; i++) {
        frame[frameIndex] = (float)spout[i];
        frameIndex = (frameIndex + 1) % frameSize;
        if (frameIndex == 0) {
            bool result = callback(frame);
            if (result == false) {
                singingSynthProcessing = false;
                goto cleanup;
            }
        }
    }

    // clear Csound spin
    memset(spin, 0, sizeof(double) * ksmps);

    //render one frame of singing voice synthesizer audio
    while (singingSynthProcessing == true &&
           singingSynthSPSCAudioQueue.size_approx() < ksmps) {
        this_thread::sleep_for(chrono::milliseconds(1));
    }

    // copy singing voice synthesizer output to Csound spin
```

```

for (int i = 0; i < ksmps && singingSynthSPSCAudioQueue.size_approx() > 0;
    i++) {
    float sample;
    bool result = singingSynthSPSCAudioQueue.try_dequeue(sample);
    spin[i] = result ? (double)sample : 0.0;
}

```

2.3 Song Renderer

The Song Renderer serves as a media encoder. It takes the raw audio output from Csound and encodes it into the Apple HTTP Live Streaming (HLS) format to be streamed across the web [6]. Designed as an Express.js application, the Song Renderer handles requests from the front-end applications. It uses a Node.js worker thread to generate music with the song composer and then render audio with the Song Engine.

This audio is then piped into a readable Node.js stream that is added as an input of an FFmpeg command via its PCM 32-bit floating-point little-endian muxer [7]. The FFmpeg command segments the audio into approximately 10-second chunks and stores the segments in MPEG-TS files, which are indexed with an extended M3U playlist file (.m3u8). These files are stored in a temporary folder, and a file watching library is used to upload them to a cloud storage service as they are created. This enables web-streaming upon the rendering of the song’s first ten seconds, providing a “near real-time” experience to the user. Upon completion, a second FFmpeg command is used to convert the HLS chunks into a single MP3 file so that the user can download the full song.

3 Deployment and Front End Integration

RIFFIT’s backend service is deployed on Google Cloud Run, a serverless compute platform that automatically scales containerized applications [8]. The service can be accessed from a variety of front ends, including various web applications and a REST API. These front ends provide a variety of supplemental features including advanced voice customization, synchronized lyric display, and deep integrations with social media and digital learning platforms.

The backend service is designed to handle consecutive requests of any genre and voice without the need for reinitialization of the Song Engine. All singing voice synthesizer models are loaded upon initialization, and the Csound API’s *csoundReset()* function is called between requests [9]. While this increases overall memory usage, it substantially reduces the system’s response time for subsequent requests and restricts CPU spikes to only the initial startup of the container. These startup spikes are counteracted by Google Cloud Run’s “Startup CPU Boost” feature, which temporarily allocates additional CPU for the startup of new containers [10]. As such, the normal CPU allocation outside of container startup is reduced substantially.

4 Conclusion

There are several reasons Csound was identified as being particularly suitable to this type of commercial text-to-song application. First, Csound’s distinct separation of the orchestra and the score lent itself to the creation of versatile instruments that can be reused by multiple genres. User defined opcodes provided the powerful extensibility required to create instruments that adapt to the user’s selected preferences. Furthermore, the text table format of the score statements facilitated easy dynamic code generation, allowing these instruments to be performed in an intelligent manner. Lastly, the Csound API provided a robust set of modules for integrating Csound into other languages. The API’s ability to manually control Csound rendering, including its audio input and output buffers, enabled crucial performance optimizations that improved both rendering latency and compute costs. The qualities outlined above provide a compelling case for the usage of Csound in commercial applications, both as a versatile compositional tool and as a reliable synthesis engine.

References

1. Canyakan, S.: The role of AI in creative processes: ethical and legal perspectives in the music industry. *Journal of Music Theory and Transcultural Music Studies* 2(2), 143-158 (2024)
2. RIFFIT,: “One Million RIFFIT Songs Created!”. *RIFFIT* (2023) <https://www.riffit.com/blog/one-million-songs>
3. Dreamtonics Co., Ltd., <https://dreamtonics.com>
4. *The Canonical Csound Reference Manual*, 6.18.0 edition, *inch* opcode. <https://csound.com/docs/manual/inch.html>
5. readerwriterqueue Github Repository. <https://github.com/cameron314/readerwriterqueue>
6. Pantos, R.: HTTP Live Streaming (RFC 8216). RFC Editor. doi:10.17487/RFC8216 (2017).
7. FFmpeg Documentation, version 8.1.2. <https://ffmpeg.org/ffmpeg-all.html>
8. Google Cloud Run, <https://cloud.google.com/run>
9. Csound API Documentation, version 6.18. *csound.h* file reference. <https://csound.com/docs/api>
10. Giannini, S: “New Startup CPU Boost Improves Cold Starts in Cloud Run, Cloud Functions”. *Google* (2022) <https://cloud.google.com/blog/products/serverless/announcing-startup-cpu-boost-for-cloud-run--cloud-functions>